

Advanced SQL for Big Data Engineers

Window functions, CTEs, time-series aggregation and query tuning

The SQL that separates a report writer from a data engineer: window functions and their frame semantics, recursive CTEs for hierarchy and lineage traversal, gaps-and-islands and sessionisation patterns, deduplication, and how to read an execution plan well enough to know why a query is slow before changing it.

Vivek Pandey — Data Engineer
Azure · Databricks · PySpark · Delta Lake
vp171097.github.io/DevWebsite

Contents

1. Window functions: the mental model
2. Ranking, and picking one row per group
3. LAG, LEAD and period-over-period change
4. Frames: ROWS vs RANGE
5. Rolling and cumulative aggregations
6. Common Table Expressions
7. Recursive CTEs: hierarchies and lineage
8. Gaps, islands and sessionisation
9. Pivoting and conditional aggregation
10. NULL semantics and join traps
11. Reading an execution plan
12. Query tuning patterns

1. Window functions: the mental model

A GROUP BY collapses rows. A window function does not — it computes a value across a set of related rows while every input row survives. That single difference removes most self-joins from analytical SQL.

```
function(...) OVER (
  PARTITION BY <cols>      -- split rows into independent groups
  ORDER BY   <cols>        -- order within each group
  <frame>                -- which rows around the current one to include
)
```

```
-- Each order keeps its own row AND sees its customer's total.
SELECT order_id, customer_id, price,
       SUM(price) OVER (PARTITION BY customer_id)           AS customer_total,
       price / SUM(price) OVER (PARTITION BY customer_id)   AS pct_of_customer,
       COUNT(*)  OVER (PARTITION BY customer_id)           AS customer_orders
FROM portfolio.ecom_silver.order_items;
```

Evaluation order Window functions run after WHERE, GROUP BY and HAVING, but before ORDER BY and LIMIT. You therefore cannot filter on a window result in the same WHERE clause — wrap it in a CTE or subquery and filter outside. This is the single most common window-function error.

2. Ranking, and picking one row per group

Function	Ties	Gaps after a tie	Typical use
ROW_NUMBER()	Broken arbitrarily	n/a	Deduplication — exactly one row per key
RANK()	Share a rank	Yes (1,1,3)	Leaderboards where ties should show
DENSE_RANK()	Share a rank	No (1,1,2)	Top-N distinct values
NTILE(n)	n/a	n/a	Bucketing into quartiles/deciles
PERCENT_RANK()	Share	n/a	Relative standing, 0 to 1

```
-- The canonical dedupe: keep the latest record per business key.
WITH ranked AS (
  SELECT *,
         ROW_NUMBER() OVER (PARTITION BY order_id
                           ORDER BY _ingested_at DESC, _source_file DESC) AS rn
  FROM portfolio.ecom_bronze.orders
)
SELECT * EXCEPT (rn) FROM ranked WHERE rn = 1;
```

Add a deterministic tiebreaker. ORDER BY _ingested_at alone gives non-deterministic results when two records share a timestamp.

```
-- Top 3 products per category, ties included.
WITH r AS (
  SELECT category_name, product_id, revenue,
         DENSE_RANK() OVER (PARTITION BY category_name ORDER BY revenue DESC) AS rnk
  FROM portfolio.ecom_gold.product_revenue
)
SELECT * FROM r WHERE rnk <= 3;
```

3. LAG, LEAD and period-over-period change

LAG and LEAD read a value from another row in the same partition without a self-join — the natural way to express growth, deltas and state transitions.

```
SELECT order_date, revenue,
       LAG(revenue, 1) OVER (ORDER BY order_date) AS prev_day,
       revenue - LAG(revenue, 1) OVER (ORDER BY order_date) AS day_delta,
       ROUND(100.0 * (revenue - LAG(revenue, 1) OVER (ORDER BY order_date))
            / NULLIF(LAG(revenue, 1) OVER (ORDER BY order_date), 0), 2) AS pct_change,
       LAG(revenue, 7) OVER (ORDER BY order_date) AS same_day_last_week
FROM portfolio.ecom_gold.agg_daily_sales
ORDER BY order_date;
```

NULLIF guards the division: without it, the first zero-revenue day divides by zero.

```
-- Detecting state changes: only rows where status actually moved.
WITH t AS (
  SELECT order_id, status, changed_at,
         LAG(status) OVER (PARTITION BY order_id ORDER BY changed_at) AS prev_status
  FROM order_status_history
)
SELECT * FROM t
WHERE prev_status IS NULL OR status <> prev_status;
```

Default values LAG(col, 1, 0) supplies 0 instead of NULL at the partition boundary. Useful for running arithmetic, misleading for reporting — a genuine 'no prior period' is not zero.

4. Frames: ROWS vs RANGE

The frame decides which rows the aggregate sees. Getting this wrong produces numbers that look plausible and are wrong, which is worse than an error.

Frame	Meaning
ROWS BETWEEN 6 PRECEDING AND CURRENT ROW	The 6 physical rows before this one, plus this one
RANGE BETWEEN 6 PRECEDING AND CURRENT ROW	All rows whose ORDER BY value is within 6 of the current value
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW	Everything from the partition start (running total)
ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING	The whole partition
RANGE BETWEEN INTERVAL 7 DAYS PRECEDING AND CURRENT ROW	A true 7-day time window, gaps included

The default frame is a trap With an ORDER BY and no explicit frame, the default is RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW. Because it is RANGE, all rows sharing the current ORDER BY value are included — so a 'running total' over a date column jumps by the whole day at once. State the frame explicitly whenever you use ORDER BY in a window.

```
-- ROWS counts rows; if a date is missing, the window silently spans 7 *rows*,
-- which may cover 10 calendar days.
AVG(revenue) OVER (ORDER BY order_date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)

-- RANGE with an interval is calendar-correct even when days are missing.
AVG(revenue) OVER (ORDER BY order_date
                   RANGE BETWEEN INTERVAL 6 DAYS PRECEDING AND CURRENT ROW)
```

5. Rolling and cumulative aggregations

```
SELECT order_date, revenue,
       -- 7-day moving average, calendar-accurate
       ROUND(AVG(revenue) OVER (ORDER BY order_date
                               RANGE BETWEEN INTERVAL 6 DAYS PRECEDING AND CURRENT ROW), 2) AS ma_7d,
       -- running total from the start of the series
       SUM(revenue) OVER (ORDER BY order_date
                          ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS cumulative,
       -- month-to-date, resets each month
       SUM(revenue) OVER (PARTITION BY DATE_TRUNC('MONTH', order_date)
                          ORDER BY order_date
                          ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS mtd,
       -- share of the month
       ROUND(100.0 * revenue
             / SUM(revenue) OVER (PARTITION BY DATE_TRUNC('MONTH', order_date)), 2) AS pct_of_mon
FROM portfolio.ecom_gold.agg_daily_sales;
```

Series with missing dates distort row-based windows. Generate a dense calendar and left-join onto it so every day exists before windowing:

```
WITH calendar AS (
  SELECT explode(sequence(DATE '2016-09-01', DATE '2018-10-31', INTERVAL 1 DAY)) AS d
)
SELECT c.d AS order_date, COALESCE(s.revenue, 0) AS revenue
FROM calendar c
LEFT JOIN portfolio.ecom_gold.agg_daily_sales s ON s.order_date = c.d;
```

6. Common Table Expressions

CTEs name a step. Their value is legibility: a five-CTE query that reads top to bottom is far easier to review than one deeply nested subquery, and each step can be run alone while debugging.

```
WITH delivered AS (
  SELECT * FROM portfolio.ecom_silver.orders WHERE status = 'delivered'
),
per_customer AS (
  SELECT customer_id,
         COUNT(*)      AS orders,
         SUM(price)     AS lifetime_value,
         MIN(purchased_at) AS first_order,
         MAX(purchased_at) AS last_order
  FROM delivered
  GROUP BY customer_id
),
scored AS (
  SELECT *,
         NTILE(4) OVER (ORDER BY lifetime_value DESC) AS value_quartile,
         DATEDIFF(CURRENT_DATE, last_order)          AS days_since_last
  FROM per_customer
)
SELECT * FROM scored
WHERE value_quartile = 1 AND days_since_last > 90;  -- valuable but lapsing
```

CTEs are not always materialized In Spark a CTE referenced twice may be computed twice. If a CTE is expensive and reused, persist it — **CACHE TABLE t AS SELECT ...** or write an intermediate table — rather than assuming the engine will reuse the result.

7. Recursive CTEs: hierarchies and lineage

A recursive CTE has an anchor (the starting rows) and a recursive part that references the CTE itself, iterating until it returns nothing. It is the right tool for org charts, bills of material, category trees and lineage graphs.

```

WITH RECURSIVE lineage AS (
  -- anchor: the table we start from
  SELECT source_table, target_table, 1 AS depth,
         CAST(source_table AS STRING) AS path
  FROM table_dependencies
  WHERE source_table = 'portfolio.ecom_bronze.orders'

  UNION ALL

  -- recursive step: everything one hop further downstream
  SELECT d.source_table, d.target_table, l.depth + 1,
         CONCAT(l.path, ' -> ', d.target_table)
  FROM table_dependencies d
  JOIN lineage l ON d.source_table = l.target_table
  WHERE l.depth < 10                -- guard against cycles
)
SELECT DISTINCT target_table, depth, path
FROM lineage
ORDER BY depth;

```

Always bound the recursion A cycle in the graph makes a recursive CTE run until it exhausts memory. A depth guard, or tracking the visited path and excluding revisits, is not optional on real-world graphs.

```

-- Organisational hierarchy with an indented label
WITH RECURSIVE org AS (
  SELECT employee_id, manager_id, name, 1 AS lvl, name AS chain
  FROM employees WHERE manager_id IS NULL
  UNION ALL
  SELECT e.employee_id, e.manager_id, e.name, o.lvl + 1,
         CONCAT(o.chain, ' / ', e.name)
  FROM employees e JOIN org o ON e.manager_id = o.employee_id
  WHERE o.lvl < 15
)
SELECT REPEAT('    ', lvl - 1) || name AS tree, lvl, chain FROM org;

```

8. Gaps, islands and sessionisation

Grouping consecutive events into runs — active streaks, user sessions, uptime windows — is the classic gaps-and-islands problem. The trick is to build a group key that only increments when a new run starts.

```
-- Sessionise events: a gap of more than 30 minutes starts a new session.
WITH flagged AS (
  SELECT user_id, event_time,
         CASE WHEN event_time - LAG(event_time) OVER (PARTITION BY user_id ORDER BY event_time)
              > INTERVAL 30 MINUTES
              OR LAG(event_time) OVER (PARTITION BY user_id ORDER BY event_time) IS NULL
              THEN 1 ELSE 0 END AS is_new_session
  FROM events
),
sessioned AS (
  SELECT *,
         SUM(is_new_session) OVER (PARTITION BY user_id ORDER BY event_time
                                   ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS session_n
  FROM flagged
)
SELECT user_id, session_no,
       MIN(event_time) AS session_start,
       MAX(event_time) AS session_end,
       COUNT(*) AS events,
       MAX(event_time) - MIN(event_time) AS duration
FROM sessioned
GROUP BY user_id, session_no;
```

The running SUM of a 0/1 flag is the standard way to manufacture a group id for consecutive runs.

```
-- Consecutive-day streaks: date minus row_number is constant within a run.
WITH d AS (
  SELECT DISTINCT user_id, DATE(event_time) AS d FROM events
),
grp AS (
  SELECT user_id, d,
         DATE_SUB(d, CAST(ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY d) AS INT)) AS streak_
key
  FROM d
)
SELECT user_id, MIN(d) AS streak_start, MAX(d) AS streak_end, COUNT(*) AS streak_days
FROM grp GROUP BY user_id, streak_key
HAVING COUNT(*) >= 5;
```

9. Pivoting and conditional aggregation

```
-- Conditional aggregation: portable and explicit
SELECT customer_id,
       SUM(CASE WHEN status = 'delivered' THEN price ELSE 0 END) AS delivered_value,
       COUNT(CASE WHEN status = 'cancelled' THEN 1 END) AS cancelled_orders,
       COUNT(DISTINCT CASE WHEN price > 500 THEN order_id END) AS high_value_orders
FROM orders GROUP BY customer_id;

-- PIVOT, when the value set is known and small
SELECT * FROM (SELECT customer_state, order_month, price FROM orders)
PIVOT (SUM(price) FOR order_month IN ('2018-01', '2018-02', '2018-03'));
```

COUNT(CASE ...) vs SUM(CASE ...) COUNT ignores NULLs, so **COUNT(CASE WHEN cond THEN 1 END)** counts matches correctly without an ELSE. Writing **ELSE 0** with COUNT counts every row — a frequent and quiet bug.

10. NULL semantics and join traps

1. **NULL = NULL is not true.** It is unknown. Use **IS NULL**, or **a IS NOT DISTINCT FROM b** for null-safe equality (Spark also offers the **<=>** operator).
2. **NOT IN with a NULL returns no rows.** If the subquery yields a single NULL, the whole predicate is unknown for every row. Prefer NOT EXISTS, or filter the NULLs out of the subquery.
3. **Aggregates skip NULLs, COUNT(*) does not.** AVG(col) divides by the count of non-null values, which is usually right — but it is not the same denominator as COUNT(*).
4. **A WHERE clause on the right table turns a LEFT JOIN into an INNER JOIN.** Put the condition in the ON clause instead.
5. **Fan-out.** Joining to a table with duplicate keys multiplies rows and inflates every SUM. Aggregate the child side first, or verify uniqueness before joining.

```
-- Trap: this silently becomes an inner join
SELECT * FROM orders o LEFT JOIN reviews r ON o.order_id = r.order_id
WHERE r.review_score >= 4;                -- drops orders with no review

-- Correct: keep unmatched rows
SELECT * FROM orders o LEFT JOIN reviews r
  ON o.order_id = r.order_id AND r.review_score >= 4;

-- Detect fan-out before trusting a total
SELECT order_id, COUNT(*) FROM order_items GROUP BY order_id HAVING COUNT(*) > 1;
```

11. Reading an execution plan

```
EXPLAIN FORMATTED
SELECT c.state, SUM(o.price)
FROM orders o JOIN customers c ON o.customer_id = c.customer_id
WHERE o.order_date >= '2018-01-01'
GROUP BY c.state;
```

What you see	What it means	What to do
BroadcastHashJoin	Small side broadcast — good	Nothing
SortMergeJoin	Both sides shuffled and sorted	Broadcast the smaller side if it fits
BroadcastNestedLoopJoin	Non-equi join, O(n*m)	Rewrite as an equi-join or pre-bucket ranges
Exchange hashpartitioning	A shuffle	Reduce it, or pre-partition the data
PushedFilters: []	Predicate not pushed to the scan	Remove UDFs/casts from the predicate
PartitionFilters: []	No partition pruning	Filter on the partition column directly

Casts defeat pruning `WHERE CAST(order_date AS STRING) LIKE '2018%'` forces a full scan. `WHERE order_date >= DATE'2018-01-01'` prunes partitions. Any function applied to the partition column on the left-hand side of a predicate blocks pruning.

12. Query tuning patterns

1. **Filter before you join.** Reducing row counts early shrinks every shuffle downstream.
2. **Project only what you need.** `SELECT *` on a wide columnar table reads columns nobody uses.
3. **Aggregate before joining** when the join is one-to-many — it avoids fan-out and shrinks the shuffle.
4. **EXISTS over IN** for large subqueries; it can short-circuit on the first match.
5. **Replace self-joins with window functions.** One pass instead of two plus a shuffle.
6. **Keep statistics current** so the optimizer chooses join strategies from facts, not guesses.
7. **Materialize expensive reused CTEs** rather than trusting the engine to cache them.
8. **Match the physical layout to the access path** — partition on the common date filter, Z-ORDER or cluster on the common high-cardinality filter.

```
-- Before: fan-out, then aggregate over an inflated row set
SELECT c.customer_id, SUM(i.price)
FROM customers c JOIN orders o ON c.customer_id = o.customer_id
                JOIN order_items i ON o.order_id = i.order_id
GROUP BY c.customer_id;

-- After: collapse the child table first, then join once per customer
WITH order_totals AS (
  SELECT order_id, SUM(price) AS order_value FROM order_items GROUP BY order_id
)
SELECT o.customer_id, SUM(t.order_value)
FROM orders o JOIN order_totals t ON o.order_id = t.order_id
GROUP BY o.customer_id;
```

Measure, then change Record the runtime and bytes scanned before the rewrite and after. A query that 'looks better' but scans the same data has not been tuned.